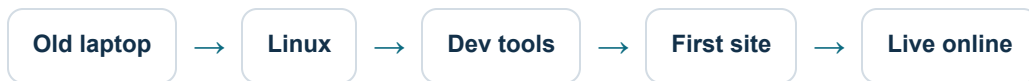


AN OPEN MIRROR PLAYBOOK

Old Laptop to Build Machine

Turn an unused laptop into a simple Linux development machine — and publish your first website.



Version 1.0 · Revised July 2026 · openmirrorllc.com

What this guide does

You have an old laptop — slow, or just sitting in a drawer. This guide turns it into a real computer for building things: writing code, making small projects, and putting real, working websites on the internet. It is written for someone who has never used Linux, a terminal, Git, or any of this before. Every term is explained the first time it appears.

By the end, you will have a laptop running Linux with the tools developers actually use, one small website you built by hand, and that website **live on the internet at a real address you can share**.

READ THIS BEFORE YOU START

Installing Linux the way this guide recommends **erases everything currently on the laptop** — its old operating system, programs, photos, and files. Chapter 2 walks through backing up anything you want to keep. Do not skip it. Nothing in this guide erases a drive automatically; the one destructive step happens inside the Linux installer, and you choose it deliberately.

What you need before you begin

- A laptop that powers on and boots reliably (its age matters less than you'd think — see Chapter 1).
- At least **2 GB of RAM** (4 GB or more is much more comfortable) and about **20 GB of free storage**.
- A USB flash drive, **8 GB or larger**, that you don't mind erasing.
- An internet connection, and the laptop's charger.
- Around 2–4 hours. You can spread it across several sittings.

THE ONE PATH THIS GUIDE SUPPORTS

To keep every instruction exact, this guide follows a single supported setup: **Linux Mint 22 (Cinnamon edition)**, which is built on Ubuntu 24.04 LTS and supported until 2029. For older or low-memory laptops, use the **Xfce edition** of the same version — every command in this guide is identical. Both cover laptops that start in either **UEFI or legacy BIOS** mode. See Chapter 3 for Secure Boot and what falls outside this guide's scope.

Contents

1. Is your laptop up to this?

2. Back up anything you want to keep

3. Install Linux

4. First boot: update and secure it

5. Set up your development tools

6. Git and GitHub: save your work

7. Build your first website

8. Put it on the internet

9. Change it, and find it later

10. What to build next

Completion checklist

Support and scope

A quick-reference terminal card, a Git & GitHub card, and a full troubleshooting & recovery guide are included as separate files in the `references/` folder — keep them open beside you.

Is your laptop up to this?

Linux runs on a much wider range of hardware than current Windows or macOS, including machines those systems have left behind. Before installing anything, check a few things so you know what to expect. (The printable *Laptop Readiness Worksheet* in the `printables/` folder is the same checklist on one page.)

Memory (RAM)

- **8 GB or more** — comfortable for everyday building.
- **4 GB** — works well with the Xfce edition; avoid dozens of browser tabs at once.
- **2 GB** — usable for the terminal-based parts, but a code editor and browser together will feel slow. Choose Xfce.

Not sure how much you have? On Windows: Settings → System → About. On a Mac: Apple menu → About This Mac. You can also search the laptop's model number online.

Storage and drive type

You need about 20 GB free. If the laptop has a solid-state drive (SSD) it will feel fast; an older spinning hard drive (HDD) still works, just slower to open programs. Neither stops you.

Does the basic hardware work?

If the laptop powers on, holds a charge, and the screen, keyboard, and trackpad work, it's ready. If something is physically broken, Linux will inherit that same problem — this guide can't fix hardware faults. You can still use an external keyboard, mouse, or monitor if one built-in part fails.

CHECKPOINT 1

The laptop powers on reliably, has at least 2 GB of RAM and ~20 GB free, and its core parts work. You've decided on the Cinnamon edition (capable laptop) or the Xfce edition (older / 4 GB or less).

Back up anything you want to keep

This is the step people regret skipping. Installing Linux will erase the laptop's internal drive. Anything you want to keep has to be copied somewhere else *first*.

1. Decide what's on the laptop

Photos and videos, documents, files you use, passwords saved in a browser, anything you'd be upset to lose. If the honest answer is "nothing I care about," you can move on. If there's anything at all, back it up.

2. Copy it somewhere that isn't this laptop

- **An external USB drive** — plug it in, copy your files over.
- **A cloud account you already use** (Google Drive, iCloud, OneDrive, Dropbox) — upload while the old system still runs.
- **Another computer** — copy over your home network or a USB drive.

Pick at least one. Two is safer than one.

3. Write down what you'll need again

Your Wi-Fi password, any logins tied only to this laptop, and browser bookmarks if they matter (most browsers can export them to a file).

4. Confirm the backup actually worked

Open the drive or cloud folder and check the files are really there. Don't just trust that the copy finished. This one check prevents almost every "I thought I backed it up" story.

CHECKPOINT 2

Everything you want to keep is copied somewhere else and you've *opened it there* to confirm. Your Wi-Fi password is written down. You're ready to erase the laptop.

Install Linux

Linux is a free operating system — the same kind of software as Windows or macOS. You don't pay for it and there's no license key. **Linux Mint** is a version of it that looks familiar and is friendly to newcomers. We'll put it on a USB stick, start the laptop from that stick, and install.

Step 1 — Download the installer

On any working computer, go to the official Linux Mint website (linuxmint.com) and download the **Cinnamon** edition (or **Xfce** for older hardware). You'll get one large file ending in `.iso`. Always download from the project's own official site — never a random mirror.

Step 2 — Make a bootable USB stick

Download **balenaEtcher** (free; works on Windows, macOS, and Linux) from its official site. Open it, select the `.iso` you downloaded, select your USB drive, and start. Everything on the USB drive is erased.

DOUBLE-CHECK THE TARGET

Before you confirm, make sure the selected drive is the *USB stick* — not your computer's main drive. This is the one moment in the whole guide to slow down and read the screen.

Step 3 — Start the laptop from the USB stick

Plug the stick into the laptop you're converting. Restart it and, as it powers on, repeatedly press the key that opens the boot menu — usually `F12` , `F2` , `Esc` , or `Del` (it varies by maker and often flashes on screen for a second). Choose to boot from the USB drive. You'll land in a "live" Linux Mint desktop running from the stick — nothing on the laptop's own drive has changed yet.

UEFI, LEGACY BIOS, AND SECURE BOOT

Linux Mint installs on laptops that start in either modern **UEFI** or older **legacy BIOS** mode — you don't need to know which yours uses. If the USB stick won't boot, look in that boot/BIOS menu for a setting called **Secure Boot** and turn it off, then try again. Linux Mint supports Secure Boot, but turning it off avoids the most common first-time snag.

Step 4 — Install, and choose the erase option deliberately

Double-click **Install Linux Mint** on the desktop. Follow the prompts for language, keyboard, and time zone. When it asks how to use the drive:

- **Erase disk and install Linux Mint** — the clean, simple choice this guide recommends for a dedicated build machine. Only pick this after finishing Chapter 2. *This is the step that erases the old system.*
- **Install alongside** (dual-boot) — keeps the old system too and lets you choose at startup. A safer option if you're not ready to fully commit the laptop, though it leaves less room for your work.

Set your name, a username, and a password you'll remember — you'll type it often, including to install software later.

Step 5 — First restart

When prompted, remove the USB stick and let the laptop restart. It should boot straight into Linux Mint.

OUTSIDE THIS GUIDE'S SCOPE

This guide supports Linux Mint 22 on a laptop with working hardware. It does not cover Wi-Fi chips that need obscure proprietary drivers, dual-boot repair, laptops that won't power on, or other Linux distributions. If your Wi-Fi doesn't work after install, connect once by ethernet cable and open **Driver Manager** from the menu — then see the Troubleshooting & Recovery reference.

CHECKPOINT 3

The laptop restarts into Linux Mint on its own, without the USB stick. You can log in with your password.

First boot: update and secure it

Connect to the internet

Click the network icon at the bottom-right and join your Wi-Fi, or plug in an ethernet cable.

Install all updates

Open **Update Manager** from the menu and install everything it offers. A fresh system usually has a lot; let it finish, then restart once more. This is the single most important thing you can do for security — it patches known problems.

Meet the terminal

The **terminal** is a text way to give the computer instructions. Open it from the menu or press `Ctrl + Alt + T`. You'll type commands and press **Enter** to run them. When a command starts with `sudo` ("do this as an administrator"), you'll be asked for your password — the characters won't appear as you type, which is normal.

READING A COMMAND

In `sudo apt update`: `sudo` = run as admin, `apt` = Mint's software installer, `update` = refresh the list of available software. The full *Terminal Reference* card in `references/` explains every command used here.

Lock screen

Confirm the screen locks automatically: **Menu** → **Screensaver**, and set it to lock after a few minutes. Combined with your login password, that keeps the machine yours.

CHECKPOINT 4

You're online, every update is installed, you've restarted, and you can open a terminal.

Set up your development tools

Now we install the tools you'll build with. Every one is free. Here's what each is for:

- **Git** — tracks changes to your work so you never lose it and can undo mistakes. Required for GitHub.
- **VS Code** — the editor where you'll write and read code.
- **Python** — a friendly programming language for scripts and backends.
- **Node.js & npm** — run JavaScript outside a browser and install its packages; most modern web tools need them.
- **GitHub CLI (`gh`)** — connect to GitHub from the terminal.
- **build-essential** — shared under-the-hood tools other software depends on.

This is a focused starter set — not everything possible, just what you need to build and ship the project ahead, and most things after it.

The easy way: run the setup script

This bundle includes a script that installs all of the above safely. Open a terminal, move into the bundle's `scripts/` folder, and run:

```
chmod +x setup-dev-machine.sh
./setup-dev-machine.sh
```

What this does: the script first prints exactly what it will install and asks you to confirm. It checks whether each tool is already present before installing it, so it's safe to run more than once. Open it in a text editor first if you'd like to read it — it's commented throughout. When it finishes, run `./verify-setup.sh` to confirm everything landed.

The manual way: install each tool yourself

If you'd rather do it by hand (and understand each piece), here are the same steps. Run them in a terminal, one block at a time.

Git, build tools, Python, and helpers:

```
sudo apt update
sudo apt install -y git build-essential python3 python3-pip python3-venv curl ca-certificates
gnupg
```

What this does: installs Git, the shared build tools, Python 3 with its package and virtual-environment helpers, and `curl / gnupg` (used in the next steps to add trusted software sources).

VS Code — from Microsoft's official software repository. (On Linux Mint, don't use "snap" for this; Mint blocks it by default. This is the official Debian/Ubuntu method.)

```
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor >
packages.microsoft.gpg
sudo install -D -o root -g root -m 644 packages.microsoft.gpg /usr/share/keyrings/microsoft.gpg
echo "deb [arch=amd64,arm64,armhf signed-by=/usr/share/keyrings/microsoft.gpg]
https://packages.microsoft.com/repos/code stable main" | sudo tee
/etc/apt/sources.list.d/vscode.list > /dev/null
rm -f packages.microsoft.gpg
sudo apt update
sudo apt install -y code
```

What this does: downloads Microsoft's signing key, registers their official VS Code software source, and installs the editor. Downloading a signing *key* is safe — it's not a program, and nothing unknown is run.

Node.js (version 22 LTS) and npm — from NodeSource's official repository, so you get a current, supported version rather than an old one.

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | sudo gpg --dearmor -o
/etc/apt/keyrings/nodesource.gpg
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg] https://deb.nodesource.com/node_22.x
nodistro main" | sudo tee /etc/apt/sources.list.d/nodesource.list > /dev/null
sudo apt update
sudo apt install -y nodejs
```

What this does: adds NodeSource's trusted source for Node.js 22 (the current long-term-support version) and installs Node.js together with npm.

GitHub CLI (`gh`) — from GitHub's official repository.

```
sudo mkdir -p -m 755 /etc/apt/keyrings
wget -nv -O- https://cli.github.com/packages/githubcli-archive-keyring.gpg | sudo tee
/etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/githubcli-archive-
keyring.gpg] https://cli.github.com/packages stable main" | sudo tee
/etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update
sudo apt install -y gh
```

Confirm it all installed

```
git --version
python3 --version
node --version
npm --version
gh --version
code --version
```

Each should print a version number. If one says `command not found`, re-run that tool's install block, or run the bundle's `verify-setup.sh` to see exactly what's missing.

CHECKPOINT 5

Git, VS Code, Python, Node.js, npm, and `gh` all print version numbers. Your machine is now a development machine.

Git and GitHub: save your work

Git is a program on your computer that saves snapshots of your work over time. **GitHub** is a website that stores copies of those snapshots online — for backup, sharing, and (in Chapter 8) publishing your site for free. Git is the tool; GitHub is one place to keep what it tracks.

Tell Git who you are

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use any name and email — this just labels your snapshots.

Create a free GitHub account

Go to github.com and sign up. It's free for public and private projects. Pick a username you don't mind being visible — it appears in your project addresses.

Sign in from the terminal

```
gh auth login
```

Choose **GitHub.com**, then **HTTPS**, and let it open your browser to finish. One-time setup per machine. (This replaces the old username-and-password method, which GitHub no longer accepts.)

The loop you'll repeat forever

```
git status          # see what changed
git add .           # stage all changes to save
git commit -m "Describe it" # save a snapshot with a note
git push            # send snapshots to GitHub
```

The *Git & GitHub Reference* card in [references/](#) keeps these commands and their meanings on one page.

CHECKPOINT 6

Git knows your name and email, you have a GitHub account, and `gh auth login` finished successfully.

Build your first website

You'll build a small, real site: a **Build Log** — one page where you type a note and a button adds it to a running list, saved right in your browser. It's small on purpose. The point isn't the project; it's finishing the whole loop: write code, run it, save it, publish it.

The finished files are in this bundle's `first-project/` folder. Type them yourself first — that's where the learning is — then compare if something misbehaves.

Step 1 — Make a project folder and open it

```
cd ~  
mkdir build-log  
cd build-log  
code .
```

What this does: moves to your home folder, makes a folder called `build-log`, moves into it, and opens it in VS Code. The three files below go in that folder — create them from VS Code's file explorer on the left.

Step 2 — The three files

Create `index.html`, `style.css`, and `script.js`. The full contents are printed in the `first-project/` `README` and files; the short version of what each does:

- `index.html` — the page's structure: a heading, a form with a text box and button, and an empty list.
- `style.css` — how it looks: fonts, spacing, colors.
- `script.js` — what it does: when you submit the form, it adds your entry to the list and saves it in the browser so it's still there next time.

Step 3 — Run it locally

A page this simple needs no server. Open the folder in your file manager and double-click `index.html`, or run `xdg-open index.html` from the terminal. Type something, press the button, and watch it appear.

IF THE BUTTON DOES NOTHING

Open the browser's developer console (F12 → Console) and look for a red error — it names the line and mistake. The most common cause is a filename in `index.html` not matching the real file, or a missing `}` in `script.js`. Compare with the files in `first-project/`.

Step 4 — Make one visible change

In `index.html`, change the big heading to something of your own. Save, reload the page in the browser, and confirm your change shows. You just edited a real website.

Step 5 — Save it with Git and push to GitHub

```
git init
git add .
git commit -m "First commit: build log page"
gh repo create build-log --public --source=. --push
```

What this does: turns the folder into a Git project, saves your first snapshot, then creates a matching GitHub repository and uploads your code — all in the last command. Use `--private` instead of `--public` to keep it hidden.

CHECKPOINT 7

Your Build Log works in a browser, you've made it your own, and the code is on GitHub at `github.com/your-username/build-log`.

Put it on the internet

"Deploying" means putting your project where anyone can reach it. We'll use **GitHub Pages** — the simplest free option for a site like this, with no extra account and no configuration file. It publishes straight from the repository you just created.

Turn on GitHub Pages

1. In a browser, open `github.com/your-username/build-log`.
2. Click **Settings**, then **Pages** in the left sidebar.
3. Under "Build and deployment," set **Source** to **Deploy from a branch**.
4. Set the branch to `main` and the folder to `/ (root)`, then **Save**.

Visit your live site

Give it a minute or two, then refresh the Pages settings — it will show "Your site is live at `https://your-username.github.io/build-log/`." Open that address. Your Build Log is now on the internet, reachable by anyone with the link.

CHECKPOINT 8

Your site loads at its `github.io` address in a normal browser. You can send that link to someone.

Change it, and find it later

The loop that makes it feel real: change something and watch the live site update.

1. In VS Code, edit `index.html` — add a line, change the heading, adjust a color in `style.css`. Save.
2. In the terminal, inside the `build-log` folder:

```
git add .  
git commit -m "Update the heading"  
git push
```

3. Wait a minute, refresh your live address — your change is there.

That loop — edit, commit, push, see it live — is exactly how real projects update, at any size.

Finding your site again later

Your live address is always `https://your-username.github.io/build-log/`. You can also find it any time under the repository's **Settings** → **Pages**. Your code lives at `github.com/your-username/build-log`, and on your laptop in the `~/build-log` folder.

WHAT GITHUB PAGES DOES AND DOESN'T DO

Pages serves static files (HTML, CSS, JavaScript) — perfect for this project. It doesn't run a live backend or database. When a later idea needs those, you'll use a different kind of host; Chapter 10 points the way.

CHECKPOINT 9

You changed the site, pushed the change, and saw it go live. You know where to find the code, the folder, and the live URL again.

What to build next

You now have a working development machine and one live project. The hardest part — finishing the whole loop once — is behind you. These free tools from Open Mirror can help you decide what to build next. None are required; they're a place to start.

- **Open Mirror** (openmirrorllc.com) — a plain, honest list of real, working things one small studio has built with the same toolset you just installed. Good for seeing finished examples.
- **StepInTheRing** (stepinthering.com) — the most useful next stop. Answer a short series of questions about an idea and it turns them into a concrete first build plan. If you're thinking "okay, but what do I build," bring that half-formed idea here. Free, and the plan is yours.
- **Reflect** (openmirrorllc.com/reflect) — a quieter tool: a few open-ended prompts if you're stuck on what you even want to make.

A good second project is usually smaller than you think. Take one idea, write down the smallest version that would truly work end to end, and build that — the same loop you just used. That's exactly what StepInTheRing helps you shape.

Completion checklist

You've finished the journey when all of these are true:

- ☐ The laptop runs Linux Mint, fully updated, with a lock screen.
- ☐ Git, VS Code, Python, Node.js, npm, and `gh` all work.
- ☐ Git knows your name and email; you're signed in to GitHub.
- ☐ You built the Build Log and made it your own.
- ☐ The code is on GitHub.
- ☐ The site is live at your `github.io` address.
- ☐ You changed it, pushed, and watched the live site update.
- ☐ You know where to find the code, the folder, and the live URL again.

Support and scope

What this guide covers: converting a working laptop to Linux Mint 22, installing a beginner development toolset, and building and deploying one static website with Git, GitHub, and GitHub Pages.

What it doesn't: repairing broken hardware, obscure Wi-Fi driver issues, other Linux distributions, dual-boot recovery, or advanced backend hosting. For problems along the way, use the *Troubleshooting & Recovery* reference in this bundle first.

This is an independent Open Mirror playbook. It isn't affiliated with Linux Mint, Microsoft, GitHub, or any other project named here; those names refer to their official free tools. Results depend on your hardware and the time you put in — the guide gives you the steps, not a guarantee.

Old Laptop to Build Machine · Version 1.0 · Revised July 2026 · An Open Mirror Playbook · openmirrorllc.com